

Security Lab Guide

Zero to Advanced — Hands-On Detection Engineering, SOAR, and Cloud Security

Personal reference · Danish Khan

Built on: Wazuh, TheHive + Cortex, LocalStack, Sigma, Terraform, Suricata

Compiled 2026-08-29

Contents

1. How to use this guide
2. Lab map: what you actually have running
3. Part 0 — Foundations (before touching a keyboard)
 1. What a SIEM actually does
 2. What Sigma is, and why it exists
 3. MITRE ATT&CK in one page
 4. SOAR, and why TheHive/Cortex exist
 5. Infrastructure as Code and why Terraform
4. Part 1 — Beginner: get everything running and look at it
 1. Starting and stopping the lab
 2. Wazuh: first login and orientation
 3. Reading your first Sigma rule
 4. Converting Sigma to KQL and to Lucene
 5. TheHive: create your first case
 6. LocalStack: your first AWS CLI commands
 7. Terraform: plan and apply your first resource
5. Part 2 — Intermediate: make the lab do something
 1. Feed Suricata alerts into Wazuh
 2. Write a Wazuh custom rule from scratch
 3. Write your own Sigma rule for a new technique
 4. Trigger Cortex analysis from a TheHive case
 5. Practice IAM detections against LocalStack
 6. Terraform: modules and remote state
6. Part 3 — Advanced: build a real detection pipeline
 1. End-to-end pipeline: Suricata → Wazuh → TheHive → Cortex
 2. Correlation rules and reducing false positives
 3. Atomic Red Team validation (safely, in an isolated VM)
 4. Detection-as-code: CI validation with GitHub Actions
 5. Threat hunting with the Wazuh/OpenSearch query DSL
 6. Where to go from here
7. Reference

1. Command cheat sheet
2. Troubleshooting log (real problems hit in this exact lab)
3. Security checklist
4. Repo map
5. Further reading

1. How to use this guide

This isn't a book to read start to finish. It's a lab manual. Each numbered section under Parts 1–3 is a hands-on exercise against services that are *already running* on your Mac right now — not simulations, not theory. Do them in order the first time through; after that, jump to whatever you're weak on.

Ground rule: every exercise here runs against your own local Docker containers or a free-tier cloud account you control. Nothing here touches a real employer's systems, and nothing here should ever be pointed at one.

2. Lab map: what you actually have running

Component	Role	URL / access	Where it lives
Wazuh	Open-source SIEM/XDR	https://localhost (dashboard) https://localhost:9200 (indexer API)	~/securitylab/wazuh-docker
TheHive	Case/alert management (SOAR)	http://localhost:9000	~/securitylab/thehive
Cortex	Automated analyzers (the "playbook" engine)	http://localhost:9001	~/securitylab/thehive
LocalStack	Open-source AWS API emulator	http://localhost:4566	~/securitylab/localstack
Suricata	Network intrusion detection (NIDS)	log files, no UI	~/securitylab/suricata
sigma-cli	Converts vendor- agnostic detection rules to KQL / Lucene	terminal (<code>sigma</code> command)	installed via pipx
gitleaks	Blocks secrets before they're committed	runs automatically on <code>git commit</code>	every repo's <code>.githooks/</code>
Terraform	Infrastructure as code (Azure)	terminal (<code>terraform</code> command)	~/security- portfolio/terraform-labs

Your portfolio repos live in `~/security-portfolio/`, each one cloned from GitHub (`danishrafiquekhan/<repo>`). Everything you build in this lab eventually gets written up and committed there — that's the whole point of the exercise: not just to learn, but to leave evidence of learning.

3. Part 0 — Foundations

LEVEL 0

3.1 What a SIEM actually does

A SIEM (Security Information and Event Management system) does three jobs: **collect** logs from many sources into one place, **normalize** them into a common shape so different log formats become comparable, and **search/alert** on patterns across that normalized data. Wazuh does all three: agents (or direct log shipping) collect events, its "decoders" normalize them, and its "rules" (XML files) alert when a pattern matches. Microsoft Sentinel does the same three jobs, just as a paid cloud service with KQL instead of Wazuh's XML rule syntax and OpenSearch storage.

The reason detection engineers write rules against a normalized schema (Sentinel calls this ASIM — Advanced Security Information Model) rather than raw logs is portability: one logical rule ("a sign-in from two impossible locations close in time") can be expressed once and applied to any log source that's been normalized into the same shape.

3.2 What Sigma is, and why it exists

Sigma is a generic, YAML-based rule format for log-based detection — the "write once" layer above vendor-specific query languages. You write a Sigma rule describing a pattern (a log source, a set of field/value conditions, and metadata like the ATT&CK technique it detects), and a converter (`sigma-cli`, which you already have installed) turns it into a real query for whatever backend you're running: KQL for Sentinel, Lucene for OpenSearch/Elasticsearch, SPL for Splunk, and so on.

This is why `detection-engineering/sigma-rules/*.yaml` in your portfolio is the valuable artifact, not the generated `.kql` file — the Sigma rule is the actual detection logic; the KQL is just one compiled target of it.

3.3 MITRE ATT&CK in one page

ATT&CK is a public knowledge base of attacker behaviour, organized as: **Tactics** (the attacker's goal, e.g. "Initial Access") each containing several **Techniques** (how they achieve it, e.g. T1078 "Valid Accounts") which may have **Sub-techniques** (T1078.004 "Cloud Accounts" specifically). Every rule you write should be tagged with the technique it detects (see the `tags:` field in your

Sigma rules) — this is what lets you build the `attack-mapping.csv` table showing coverage, and it's the shared vocabulary interviewers expect you to speak.

3.4 SOAR, and why TheHive/Cortex exist

A SIEM tells you something happened. A SOAR (Security Orchestration, Automation and Response) platform is where a human (or automation) then *does something about it*: opens a case, enriches it with context (is this IP known-bad? has this user done this before?), and takes action (disable the account, block the IP). TheHive is the case-management half; Cortex is the automation/enrichment half — a Cortex "analyzer" is the open-source equivalent of a single step in a Sentinel Logic App playbook.

3.5 Infrastructure as Code, and why Terraform

Instead of clicking through a cloud portal to create resources (which isn't repeatable, reviewable, or version-controlled), you write a declarative description of the desired state (`.tf` files) and a tool reconciles reality to match it. `terraform plan` shows you the diff before anything changes; `terraform apply` executes it; `terraform destroy` tears it back down. This matters for a security role specifically because most real security infrastructure (Sentinel workspaces, log pipelines, detection rules) is deployed this way in mature teams — "detection as code."

4. Part 1 — Beginner: get everything running and look at it

LEVEL 1

4.1 Starting and stopping the lab

You don't need every stack running all the time — they use real CPU/RAM/disk. Start only what you're using:

```
cd ~/securitylab/wazuh-docker/single-node && docker compose up -d # SIEM
cd ~/securitylab/thehive && docker compose up -d # SOAR
cd ~/securitylab/localstack && docker compose up -d # AWS emulator
cd ~/securitylab/suricata && docker compose up -d # NIDS

# stop (keeps data), from any of the same folders:
docker compose stop

# fully remove containers (data in named volumes survives):
docker compose down

# check what's running right now:
docker ps
```

Watch your disk. These stacks together can use 10–15GB. Run `df -h /` periodically; if it's tight, stop stacks you're not actively using, or `docker system prune` to reclaim unused image layers.

4.2 Wazuh: first login and orientation

1. Open <https://localhost> (accept the self-signed cert warning — it's your own local CA, not a real security risk on localhost).
2. Log in with `admin` / `SecretPassword` — then immediately go to your user menu and change it. This default is published in Wazuh's own docs; leaving it is the one genuinely bad habit this lab could teach you.
3. Look at the left-hand menu: **Threat Hunting** (search all indexed alerts), **Security Events**, **Integrity Monitoring**, **Vulnerability Detection**. Right now these are mostly empty — you haven't fed it any agent data yet. That's Part 2.
4. Try the search bar in Threat Hunting with a wildcard query `*` to see the manager's own internal events (it monitors itself) — this proves the pipeline (manager → indexer →

dashboard) works end to end before you add real log sources.

4.3 Reading your first Sigma rule

Open `~/security-portfolio/detection-engineering/sigma-rules/impossible-travel.yml`.

Read it top to bottom:

- `logsource` — what kind of log this rule expects (category/product/service)
- `detection.selection` — the actual condition(s), as field/value pairs
- `detection.condition` — how the named blocks combine (here, just `selection` alone)
- `falsepositives` — be honest here; every real rule has some
- `level` — severity, used for triage priority

4.4 Converting Sigma to KQL and to Lucene

```
cd ~/security-portfolio/detection-engineering
export PATH="$HOME/.local/bin:$PATH"

# regenerate every KQL query from the Sigma sources
make convert
cat kql-conversions/generated/impossible-travel.kql
```

Try converting a rule to a different backend yourself, to see the same logic expressed differently:

```
sigma convert -t opensearch_lucene -p ecs_windows sigma-rules/impossible-travel.yml
```

If that errors about a missing table/pipeline mapping, that's expected and instructive — see §9.2 (Troubleshooting) for exactly why, and read the "What I learned" section in `detection-engineering/README.md`, which documents the real fix used for the KQL conversion.

4.5 TheHive: create your first case

1. Open <http://localhost:9000>. First run walks you through creating the initial organisation and admin account — there's no default login, by design.
2. Create a case manually: **Cases** → **New Case**. Title it something like "Suspicious sign-in — T1078.004 (practice)". Add a description, severity, and a tag matching the ATT&CK technique.
3. Add an **Observable** to the case (e.g. an IP address, even a fake one like `198.51.100.23` — a real reserved-for-documentation range). This is the object Cortex will later enrich.

4.6 LocalStack: your first AWS CLI commands

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1

aws --endpoint-url=http://localhost:4566 s3 mb s3://my-lab-bucket
aws --endpoint-url=http://localhost:4566 s3 ls
aws --endpoint-url=http://localhost:4566 iam create-user --user-name lab-user
aws --endpoint-url=http://localhost:4566 iam list-users
```

Tip: put the endpoint and fake credentials in a shell alias or a `.envrc` so you're not retyping them every time.

4.7 Terraform: plan and apply your first resource

```
cd ~/security-portfolio/terraform-labs/01-first-resource-group
az login                                # opens a browser to authenticate
az account list --output table          # confirm you're on your free-tier subsc
cp terraform.tfvars.example terraform.tfvars
# edit terraform.tfvars: put your real subscription ID in

terraform init
terraform plan                          # read this output carefully before apply
terraform apply                         # type "yes" when prompted
# ... when done exploring:
terraform destroy
```

Set a spending cap on the Azure subscription *before* running `apply` — see the Security Checklist (§9.3).

5. Part 2 — Intermediate: make the lab do something

LEVEL 2

5.1 Feed Suricata alerts into Wazuh

Right now Suricata and Wazuh run independently. Wiring them together is the first real "pipeline" exercise:

1. Confirm Suricata is writing alerts: `cd ~/securitylab/suricata && docker compose up -d`, then check `suricata/logs/eve.json` is growing.
2. Wazuh natively understands Suricata's `eve.json` format. Add a `<localfile>` block to the Wazuh manager's `ossec.conf` pointing at that file, or mount the Suricata logs volume into the Wazuh manager container so it can read them directly.
3. Restart the manager, generate a test alert (Suricata ships example rules that fire on benign traffic like ICMP ping), and confirm it shows up in Wazuh's **Threat Hunting** view.

This is genuinely fiddly the first time — expect to spend real time on file paths and container networking. That struggle is the learning.

5.2 Write a Wazuh custom rule from scratch

Wazuh rules are XML, not Sigma (Sigma isn't a natural fit for host/network telemetry the way it is for cloud logs — see [detection-engineering/local-lab/README.md](#) for why). A minimal custom rule, added to `/var/ossec/etc/rules/local_rules.xml` inside the manager container:

```
<group name="local,suricata,">
  <rule id="100010" level="10">
    <if_sid>86601</if_sid> <!-- base Suricata alert rule -->
    <field name="alert.severity">1</field>
    <description>High-severity Suricata alert detected</description>
  </rule>
</group>
```

Restart the manager after editing rules (`docker exec <container> /var/ossec/bin/wazuh-control restart`) and confirm the new rule ID appears when a matching event comes through.

5.3 Write your own Sigma rule for a new technique

Pick an ATT&CK technique not yet covered (check `attack-mapping.csv`). Good next candidates for the Azure-AD-signin logsource you already have working: **T1110** (Brute Force — repeated failed sign-ins from one source), or **T1098** (Account Manipulation — new MFA method registered). Write the YAML, run it through `sigma check` to validate syntax, then `make convert`.

5.4 Trigger Cortex analysis from a TheHive case

Link Cortex to TheHive (one-time): in TheHive's admin settings, add Cortex as a "Responder/Analyzer" server using Cortex's URL (`http://cortex:9001` from inside the Docker network) and an API key generated in Cortex's UI (**Organization** → **API Keys**). Once linked, open the observable you added in §4.5 and run an available analyzer against it. Community-edition analyzers that need no external API key are limited, but this proves the automation wiring works — the same mechanism a paid analyzer (e.g. VirusTotal, needs your own free API key) would use.

5.5 Practice IAM detections against LocalStack

Simulate a plausible attack pattern and think about how you'd detect it:

```
aws --endpoint-url=http://localhost:4566 iam create-user --user-name shadow-admin
aws --endpoint-url=http://localhost:4566 iam attach-user-policy \
  --user-name shadow-admin --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
aws --endpoint-url=http://localhost:4566 iam create-access-key --user-name shadow-a
```

That sequence — create user, immediately attach full-admin policy, immediately mint an access key — is a real, well-known privilege-escalation/persistence pattern. Write up (in `aws-identity-detection/`) what a CloudTrail-based Sigma rule for this sequence would look for, even though you can't fire it against real CloudTrail in the free tier (see §9.2).

5.6 Terraform: modules and remote state

Move on to `terraform-labs/02-remote-state-azure-backend` and `03-modules-networking-compute`. The core new idea in each: state (what Terraform thinks exists) should live somewhere shared and locked (an Azure Storage blob, not your laptop disk) once more than one person or machine touches it; and repeated infrastructure patterns belong in a reusable module rather than copy-pasted resource blocks.

6. Part 3 — Advanced: build a real detection pipeline

LEVEL 3

6.1 End-to-end pipeline: Suricata → Wazuh → TheHive → Cortex

This is the capstone exercise that ties every piece in this lab together, and the strongest single thing you can describe in an interview:

1. Suricata detects network-level badness and writes to `eve.json` (from §5.1).
2. Wazuh ingests it, and your custom rule (§5.2) fires an alert above a severity threshold.
3. Configure Wazuh's **active response** or a simple script to POST that alert to TheHive's API, auto-creating a case (TheHive's REST API: `POST /api/v1/case`).
4. Cortex automatically enriches the new case's observables (the source IP, say) the moment it's created, via a pre-configured responder.

You will hit real integration friction here (auth tokens, API payload shapes, container-to-container networking). Document each obstacle and fix in the relevant repo's README, in the same style as the existing "What I learned" sections — that documentation *is* the portfolio artifact, arguably more than the pipeline itself.

6.2 Correlation rules and reducing false positives

A single-event rule (like both current Sigma rules) fires on one log line. A correlation rule fires on a *pattern across multiple events over time* — true "impossible travel" (same user, two countries, N minutes apart) needs this. Sigma's newer spec supports `correlation` rule types (temporal proximity, value counts); Wazuh supports this natively via its own `<frequency>` / `<timeframe>` rule options. Rewriting `impossible-travel.yml` as a genuine correlation rule (rather than today's proxy — alerting on Identity Protection's own risk flag) is the single best "advanced" upgrade available in this repo.

6.3 Atomic Red Team validation (safely, in an isolated VM)

Follow `atomic-red-team-validation/LAB-SETUP.md` exactly — UTM, NAT-only networking, snapshot before every session, never on this Mac directly. Once the Windows VM is built: run a test for a technique you have a detection for (T1078.004 is the one already scaffolded), and check whether your rule actually fires on the resulting telemetry. This is where "I wrote a detection"

becomes "I proved a detection works," which is the entire point of the `atomic-red-team-validation` repo.

6.4 Detection-as-code: CI validation with GitHub Actions

Add a workflow to `detection-engineering/.github/workflows/validate.yml` that runs `sigma check` (and ideally `make convert`) on every push, so a syntactically broken rule can never merge silently. This is the specific "detection as code" signal the original study plan called out as a differentiator — small to add, meaningfully raises the bar of the repo.

6.5 Threat hunting with the Wazuh/OpenSearch query DSL

Once you have real data flowing (from §5.1/6.1), practice hunting rather than just waiting for alerts: use Wazuh's Threat Hunting search with Lucene query syntax (`rule.level:>=10 AND data.srcip:*`) or drop into the raw OpenSearch API directly:

```
curl -sk -u admin:<your-new-password> \
  "https://localhost:9200/wazuh-alerts-*/_search?pretty" \
  -H 'Content-Type: application/json' -d '{
    "query": { "range": { "rule.level": { "gte": 10 } } },
    "sort": [{ "@timestamp": "desc" }],
    "size": 5
  }'
```

6.6 Where to go from here

- Add a real free-tier **Azure AD tenant** (separate from any work account) so the Sentinel/KQL rules can eventually run against real (your own) sign-in data, not just convert cleanly.
- Look at **Shuffle** (open-source, also free) as a second SOAR option alongside TheHive/Cortex — different UX, same underlying idea, worth knowing both since job postings vary.
- Revisit `intune-endpoint-health-platform` — migrate your existing, already-built project into the portfolio repo; it's real production-adjacent work and shouldn't be the least-developed repo.

9. Reference

9.1 Command cheat sheet

```
# --- Docker stacks ---
docker compose up -d          # start (from the stack's own folder)
docker compose stop           # stop, keep data
docker compose down           # remove containers, keep named volumes
docker ps                     # what's running
docker stats --no-stream      # resource usage snapshot
docker system prune           # reclaim disk from unused layers

# --- Sigma ---
export PATH="$HOME/.local/bin:$PATH"
sigma check sigma-rules/*.yml
sigma convert -t kusto -p kql-conversions/pipelines/signinlogs-table.yml -p sentinel
make convert                  # regenerate all KQL from Sigma sources

# --- Terraform ---
az login
terraform init
terraform plan
terraform apply
terraform destroy

# --- LocalStack / AWS CLI ---
export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1
aws --endpoint-url=http://localhost:4566 <any aws command>

# --- git / gitleaks ---
git config core.hooksPath .githooks      # one-time, per clone
git commit -m "..."                      # gitleaks runs automatically, blocks on sec
```

9.2 Troubleshooting log (real problems hit in this exact lab)

Symptom	Cause	Fix
sigma convert: "Unable to determine table name from rule"	The <code>sentinel_asim</code> pipeline only auto-maps a few built-in categories (e.g. <code>network_connection</code>);	Custom pipeline YAML using <code>set_state</code> to set <code>query_table: SigninLogs</code> , at a <i>lower</i> priority number than <code>sentinel_asim</code> so it runs first — see

	<code>authentication / signinlogs</code> isn't one	<code>kql-conversions/pipelines/signinlogs-table.yml</code>
Wazuh cert generator: "Permission denied" on <code>root-ca-manager.pem</code>	The generator <code>chmods</code> its output directory read-only partway through, before writing the manager-cluster cert pair	<code>chmod 755</code> the certs directory, then manually copy <code>root-ca.pem/key</code> to <code>root-ca-manager.pem/key</code> — scripted in <code>setup-wazuh.sh</code>
TheHive container exits immediately: "application secret has not been set"	<code>--no-config-secret</code> doesn't disable the check in prod mode, despite the name	Generate a real secret (<code>openssl rand -hex 32</code>), pass via <code>.env</code> + <code>--secret</code> , never hardcode it
LocalStack: CloudTrail service not listed / not usable	CloudTrail emulation is a Pro-only (paid) LocalStack feature	None available for free — practice IAM/S3/STS-based detections instead, or use a real AWS free-tier account for actual CloudTrail
gitleaks didn't catch an obviously secret-looking string	The default <code>generic-api-key</code> rule matches secret values against a fixed character set; some symbols (e.g. <code>\$</code>) fall outside it	Don't treat the hook as a guarantee — it's a backstop. The real control is never writing real credentials to a file in the first place (env vars, CLI login flows)
Wazuh/TheHive/Cortex all pull "linux/amd64" images on Apple Silicon	These projects don't yet publish arm64 builds	They run correctly under Docker's x86 emulation, just slower to start — budget extra time, don't assume a stuck-looking container is broken

9.3 Security checklist

- 2FA enabled on GitHub (and every cloud account you create for this lab)
- Azure/AWS: spending alert/budget cap set *before* creating any resource
- Terraform auth always via `az login` 's short-lived token — never a long-lived key committed anywhere
- `git config core.hooksPath .githooks` run on every fresh clone (gitleaks doesn't activate on its own)
- Atomic Red Team: VM only, NAT/host-only networking, never bridged, snapshot before every session
- All screenshots/log excerpts sanitised (device names, usernames, IPs) before committing
- Change Wazuh's default dashboard password on first login
- Run `terraform destroy` when done with an exercise so nothing keeps costing money unattended

9.4 Repo map

Repo	What lives here
<code>detection-engineering</code>	Sigma rules, KQL conversions, ATT&CK mapping, Wazuh local-lab setup
<code>atomic-red-team-validation</code>	Attack simulation case studies, VM safety setup
<code>sentinel-soar-playbooks</code>	Sentinel Logic App playbooks, TheHive/Cortex local-lab setup
<code>aws-identity-detection</code>	CloudTrail/IAM detections, LocalStack local-lab setup
<code>intune-endpoint-health-platform</code>	Existing Intune project (to be migrated in)
<code>terraform-labs</code>	Terraform Associate study exercises on Azure
<code>danishrafiquekhan</code>	GitHub profile README

9.5 Further reading

- Sigma rule specification & SigmaHQ public rule repository — github.com/SigmaHQ
- MITRE ATT&CK — attack.mitre.org
- Wazuh documentation — documentation.wazuh.com
- TheHive/Cortex documentation — docs.strangebee.com
- LocalStack documentation — docs.localstack.cloud
- Terraform Associate exam guide — developer.hashicorp.com/terraform/tutorials/certification-003
- Atomic Red Team — github.com/redcanaryco/atomic-red-team